Inference Basics

CSE 585 Advanced Scalable Systems for Agentic AI

Feb 9, 2026

Shiqi He

Adapted from materials originally created by
Jae-Won Chung

Shiqi He

- Bio
  - 3rd year PhD student working with Mosharaf
  - https://tctower.github.io/
- Background
  - 2020 – 2023: Efficient Machine Learning, Federated Learning
  - 2023 – now: Agentic AI Systems, Efficient Multimodal Models, Diffusion Serving
- Three lectures
  - Jan 15 (Thu) - Systems for AI Basics
  - Jan 20 (Tue) - Distributed Training Basics
  - Feb 10 (Tue) - Inference Basics

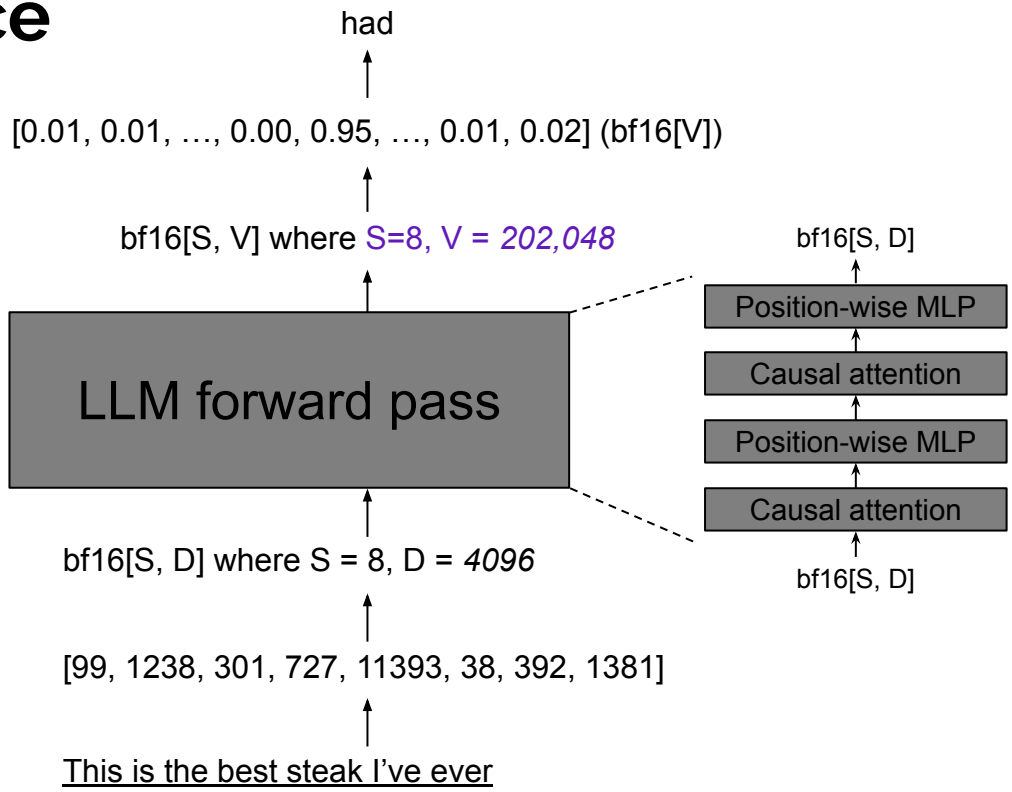
Agenda

- LLM Inference
  - KV Cache
  - Prefill & Decode
- Performance Metrics
  - TTFT, TPOT, etc
  - Arithmetic Intensity
- Optimizations
  - Orca (OSDI ’22)
  - vLLM (SOSP ’23)
- More System Challenges
  - Evaluating LLM Inference Systems
- Please ask questions if you don’t get something

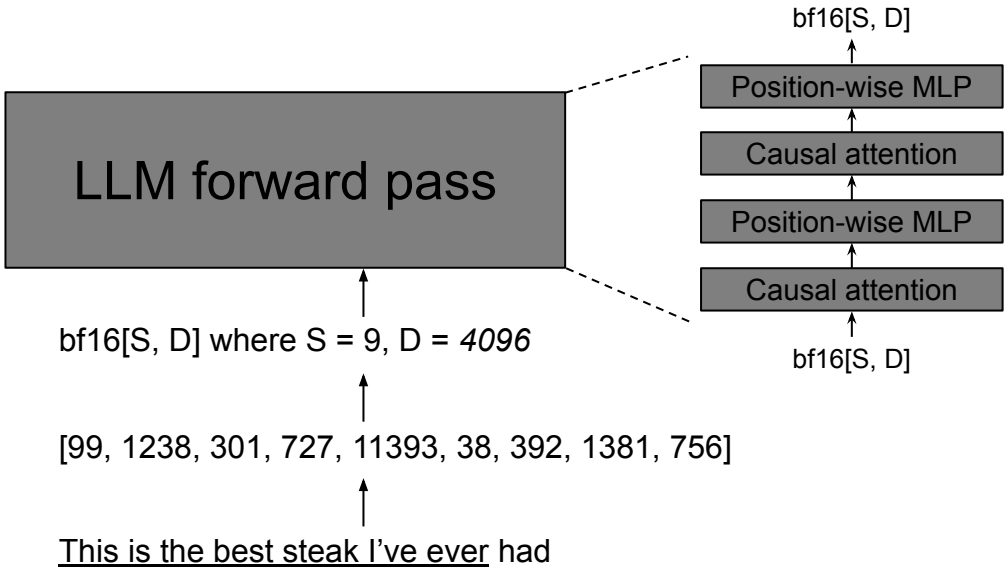
Agenda

- LLM Inference
  - KV Cache
  - Prefill & Decode
- Performance Metrics
  - TTFT, TPOT, etc
  - Arithmetic Intensity
- Optimizations
  - Orca (OSDI ’22)
  - vLLM (SOSP ’23)
- More System Challenges
  - Evaluating LLM Inference Systems
- Please ask questions if you don’t get something

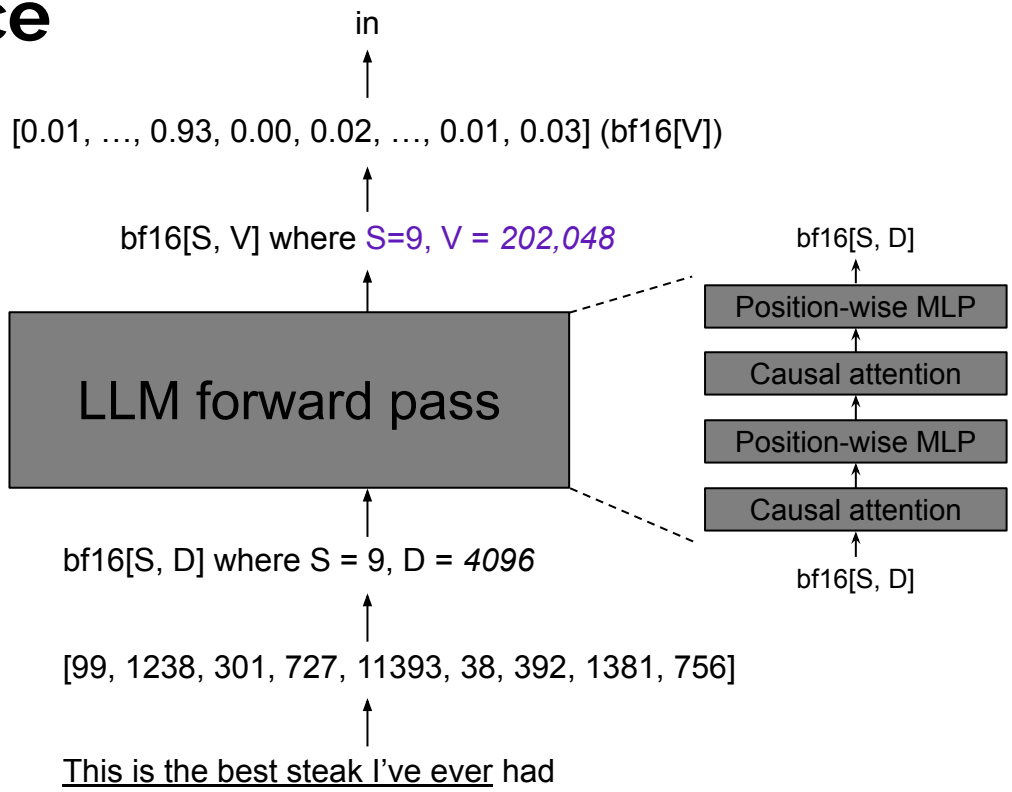
LLM Inference



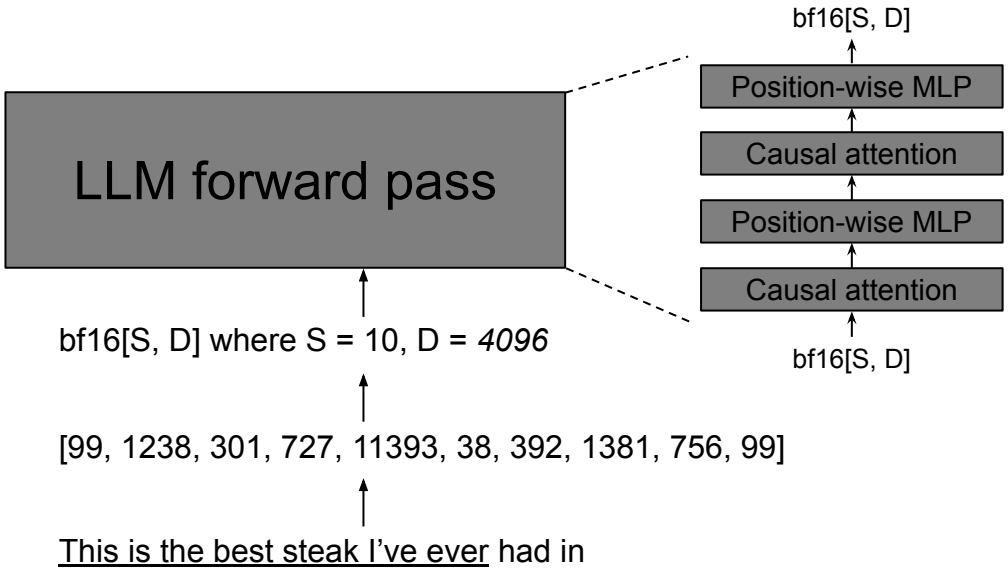
LLM Inference



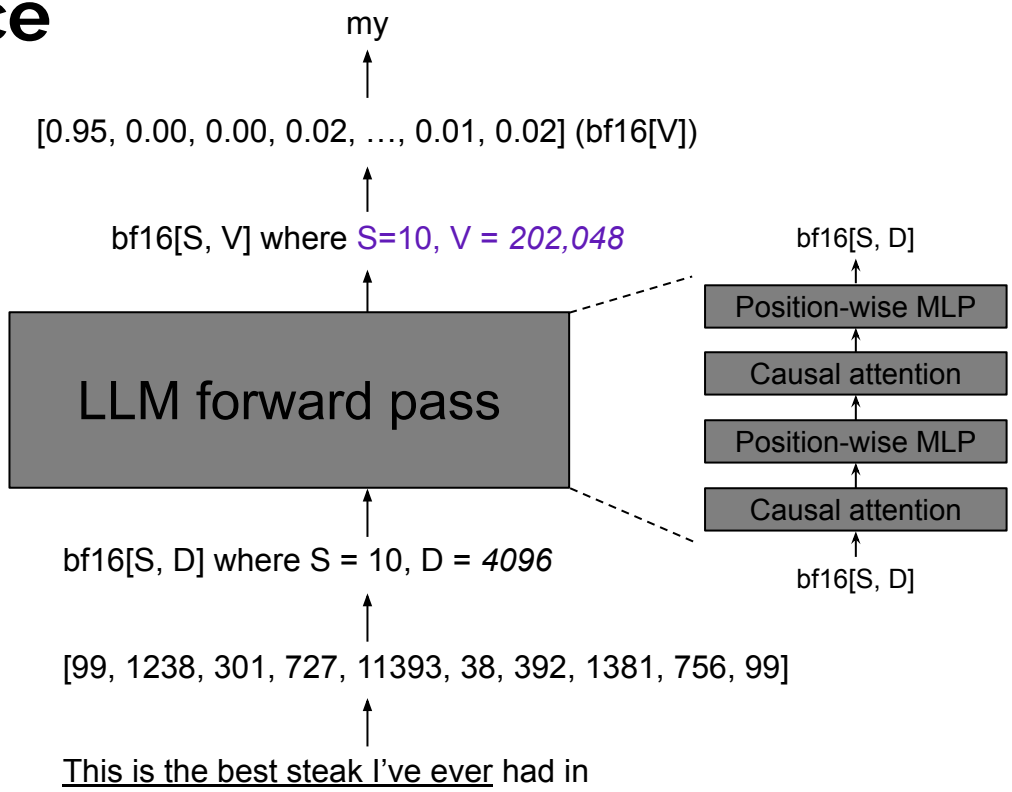
LLM Inference



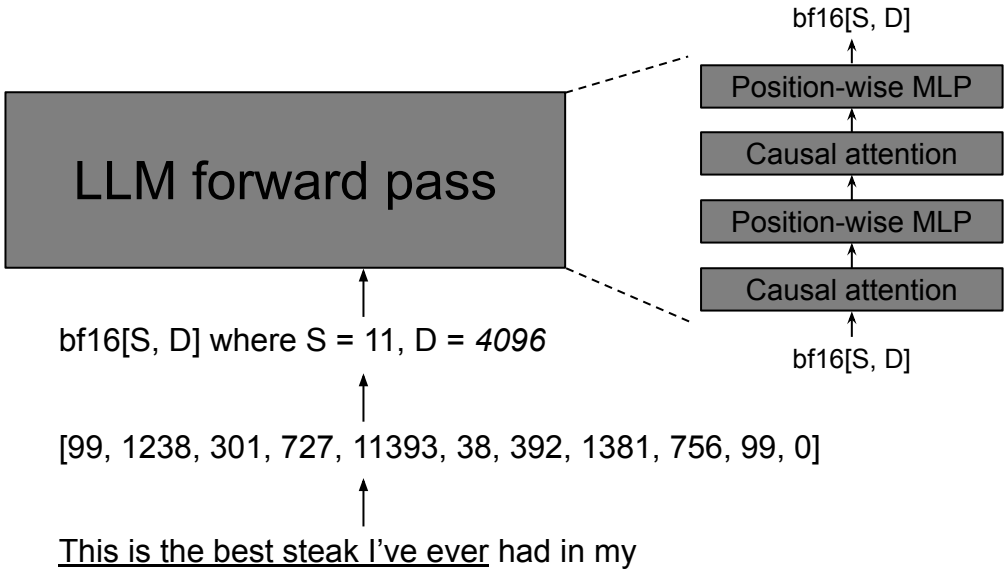
LLM Inference



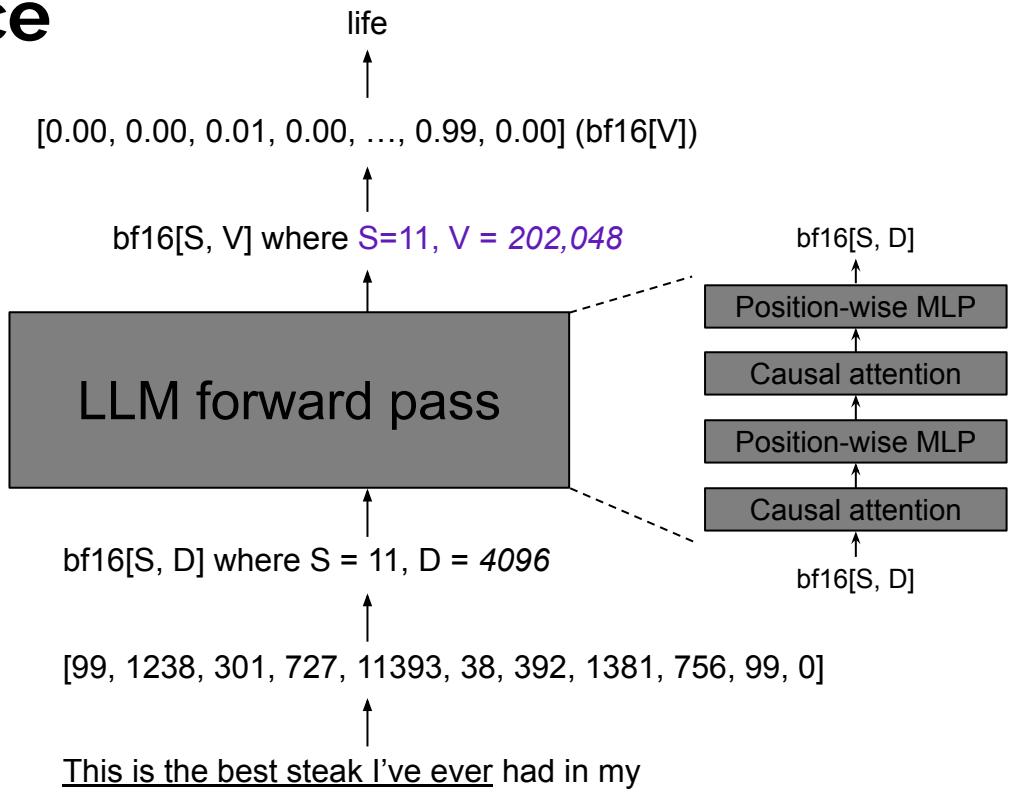
LLM Inference



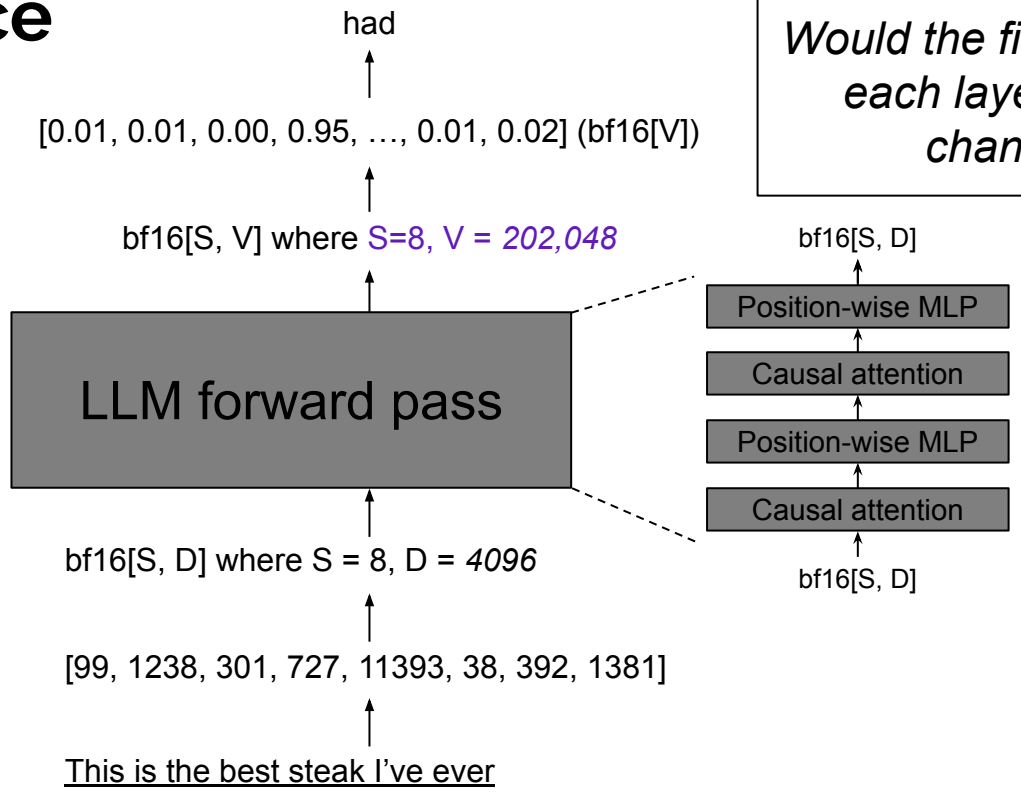
LLM Inference



LLM Inference

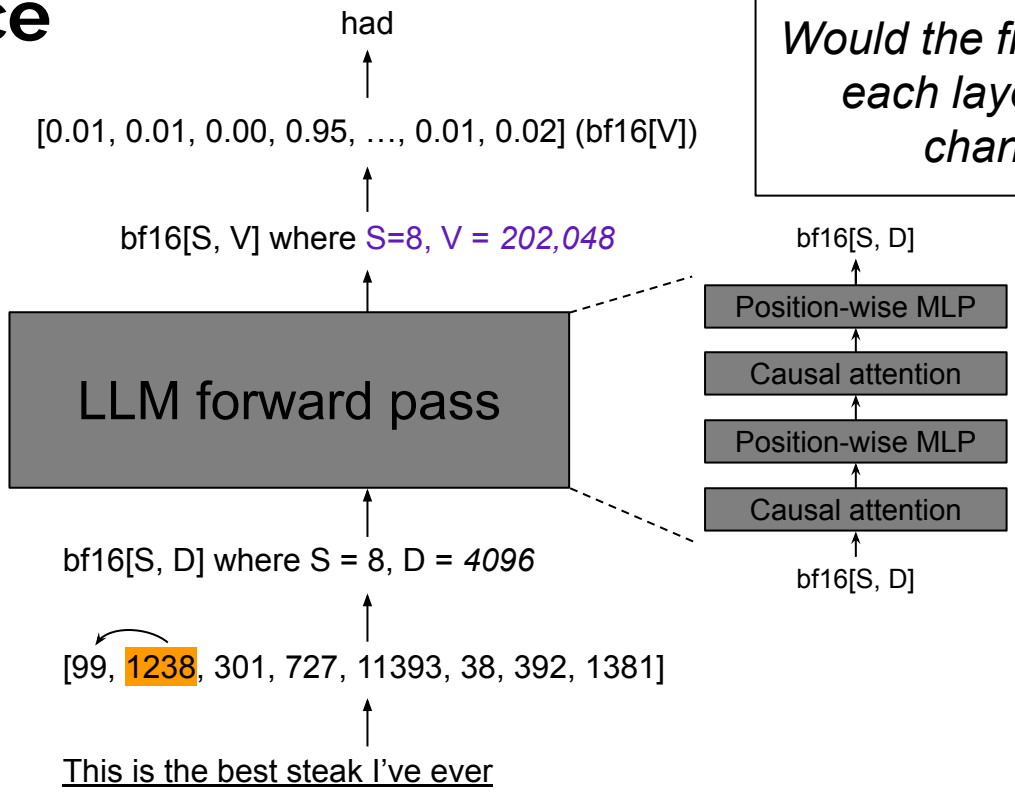


LLM Inference



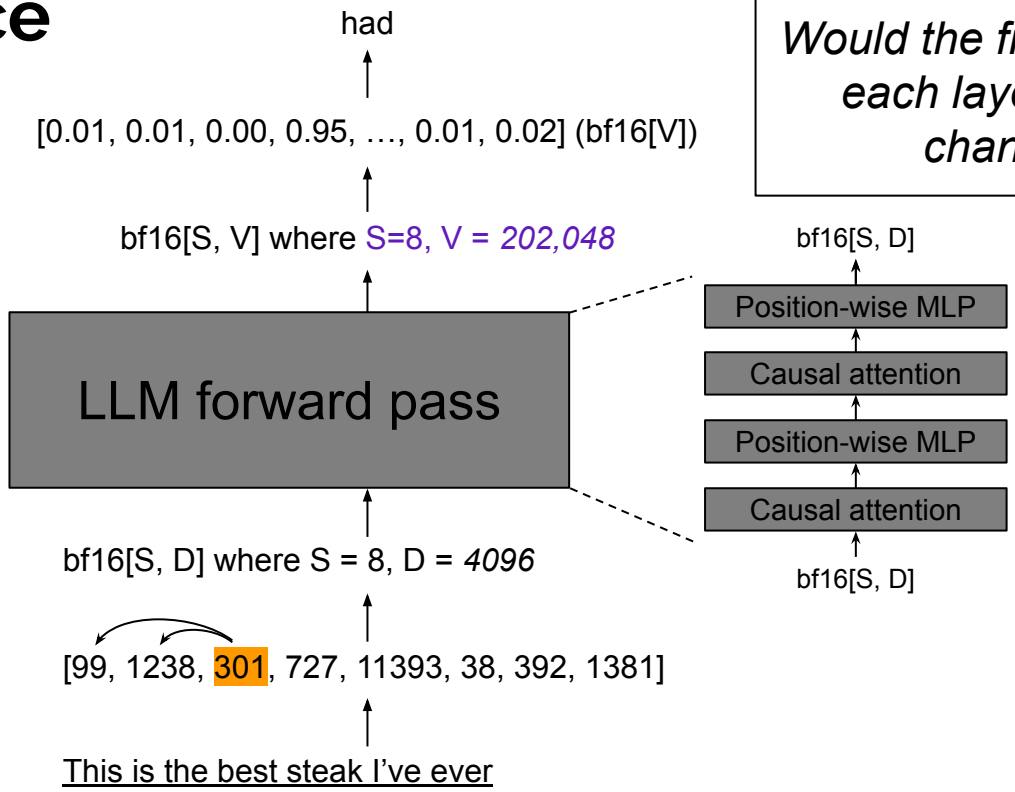
Would the first 8 positions of each layer's activation change at all?

LLM Inference



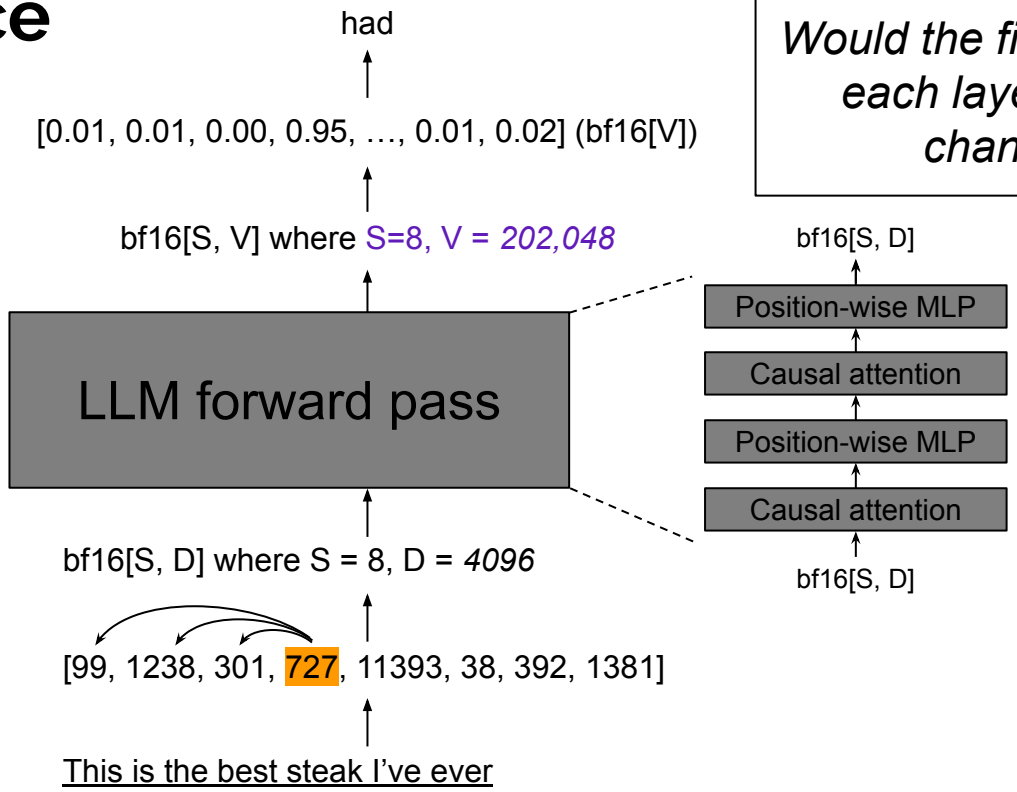
Would the first 8 positions of each layer's activation change at all?

LLM Inference



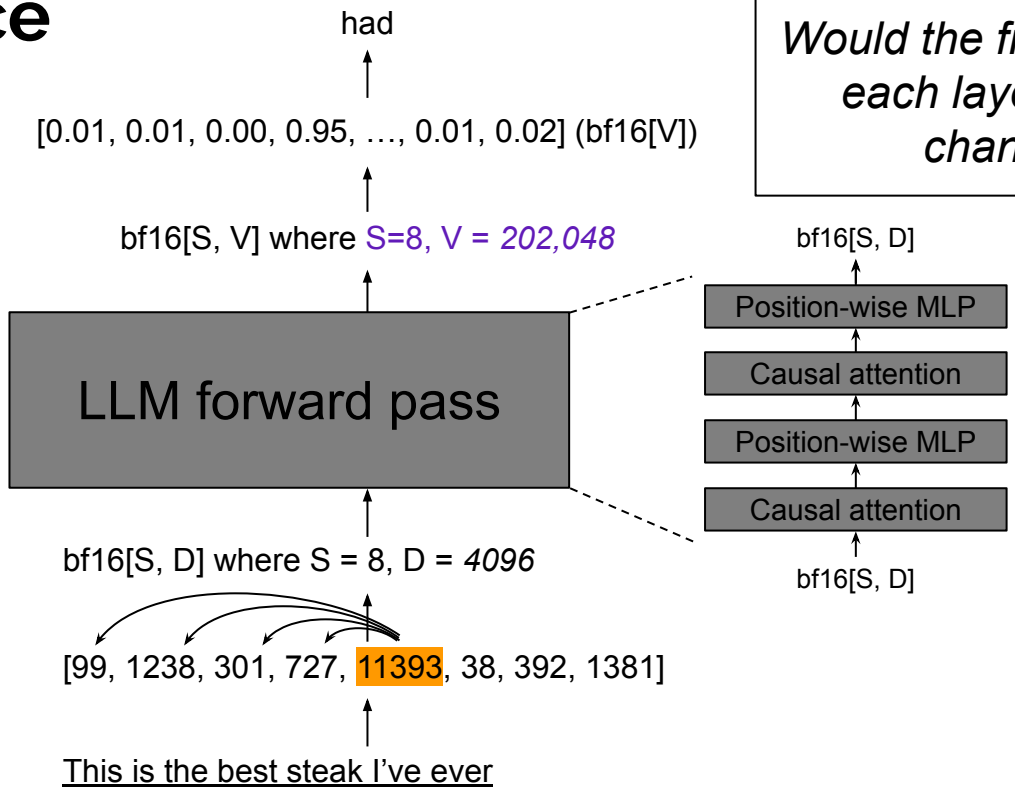
Would the first 8 positions of each layer's activation change at all?

LLM Inference



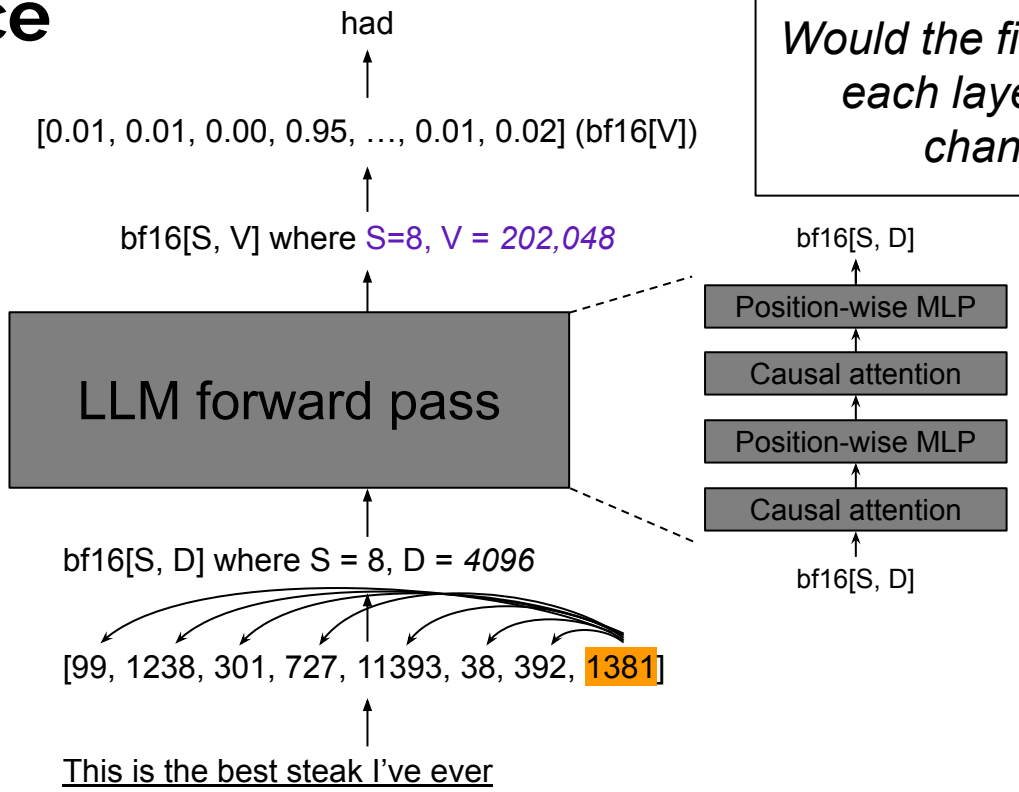
Would the first 8 positions of each layer's activation change at all?

LLM Inference



Would the first 8 positions of each layer's activation change at all?

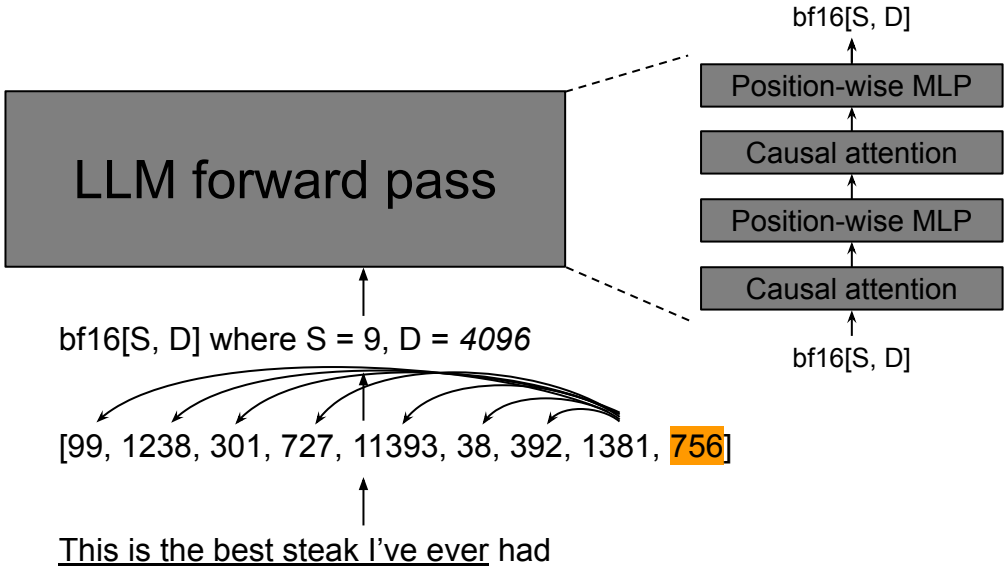
LLM Inference



Would the first 8 positions of each layer's activation change at all?

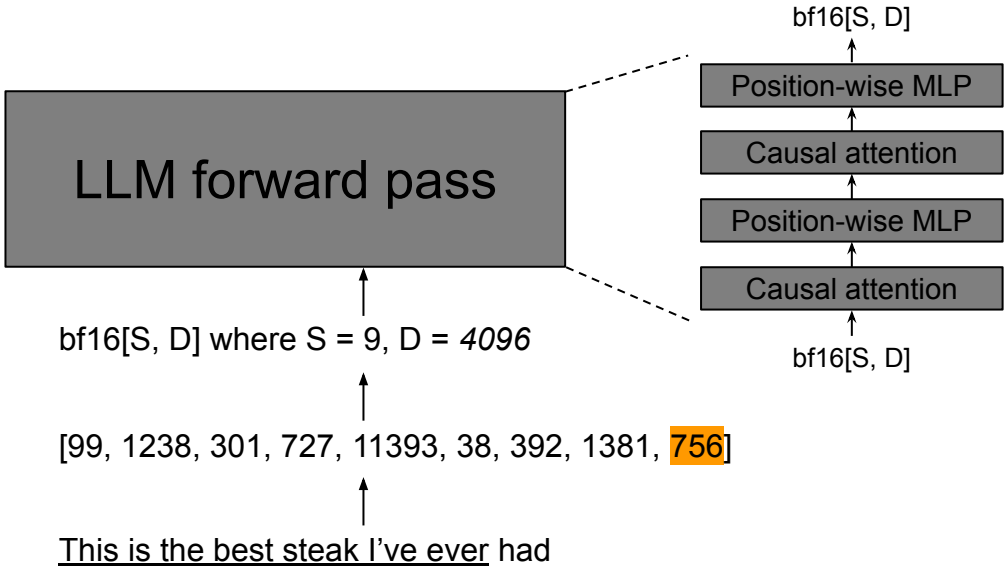
LLM Inference

*The first 8 positions of intermediate activations are **the same**.*



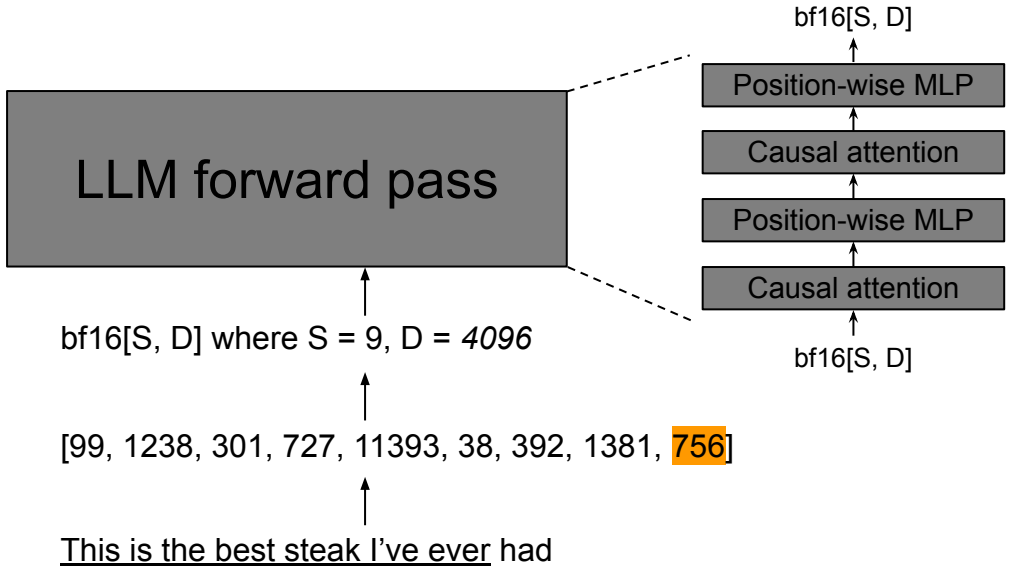
LLM Inference

*Re-computing earlier positions seems super **redundant**.*

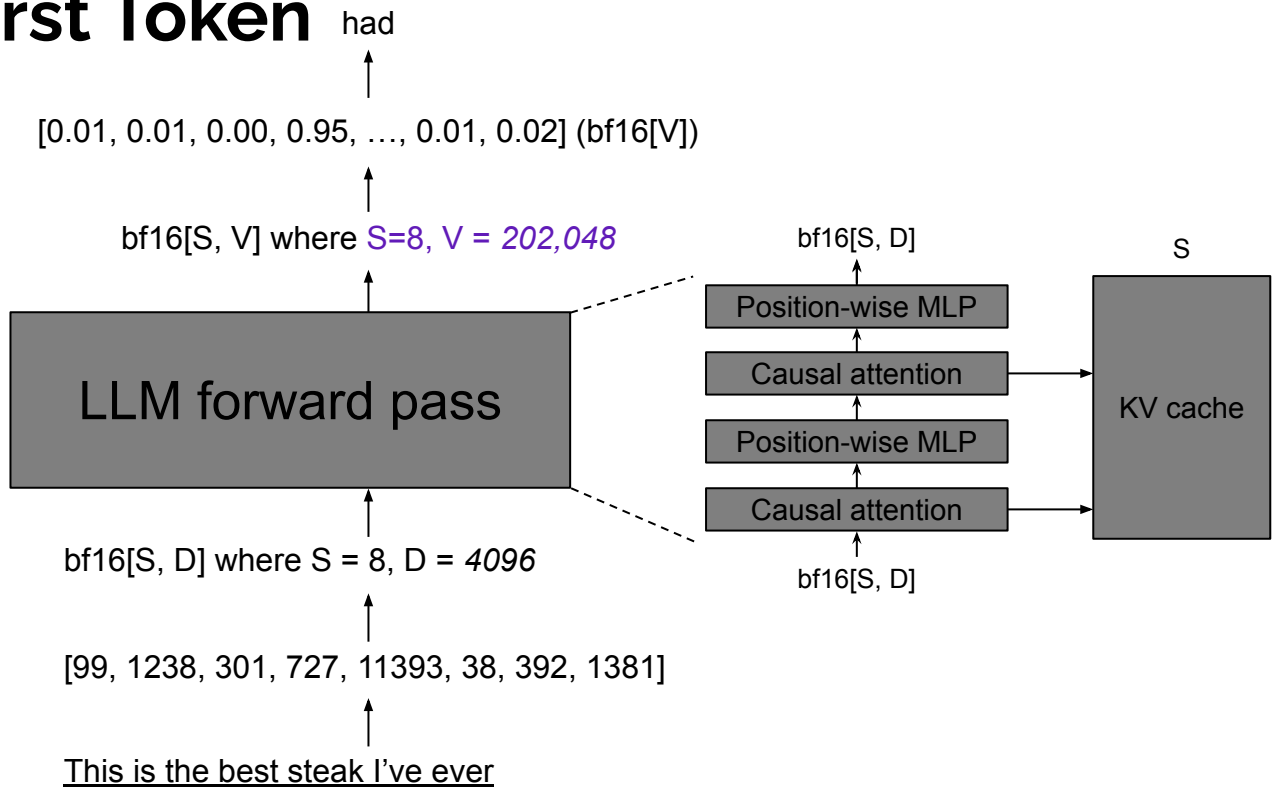


Reducing Computation for the Second+ Tokens

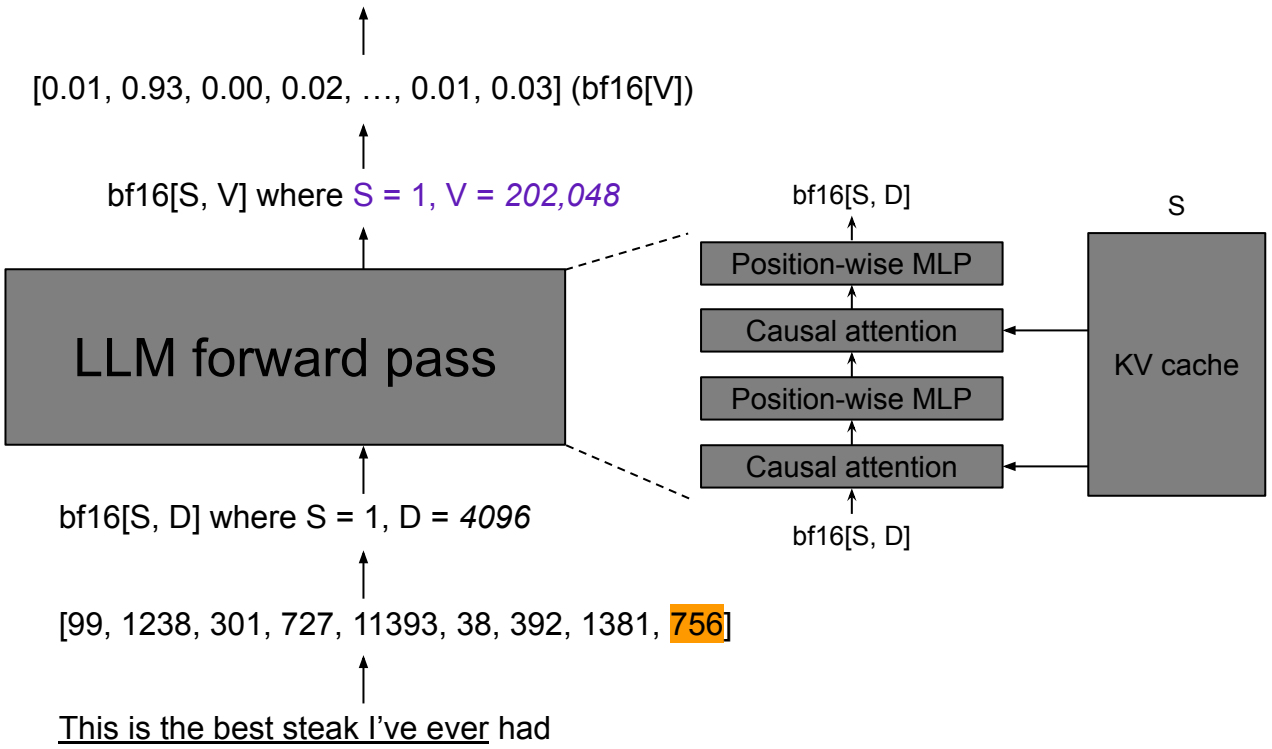
- Position-wise MLP
  - As long as you get the bf16[D] activations for the 9th position, the MLP can be computed
- Causal attention
  - It *does* depend on earlier tokens
  - You just needs two things for each token position:
    - Key tensor
    - Value tensor



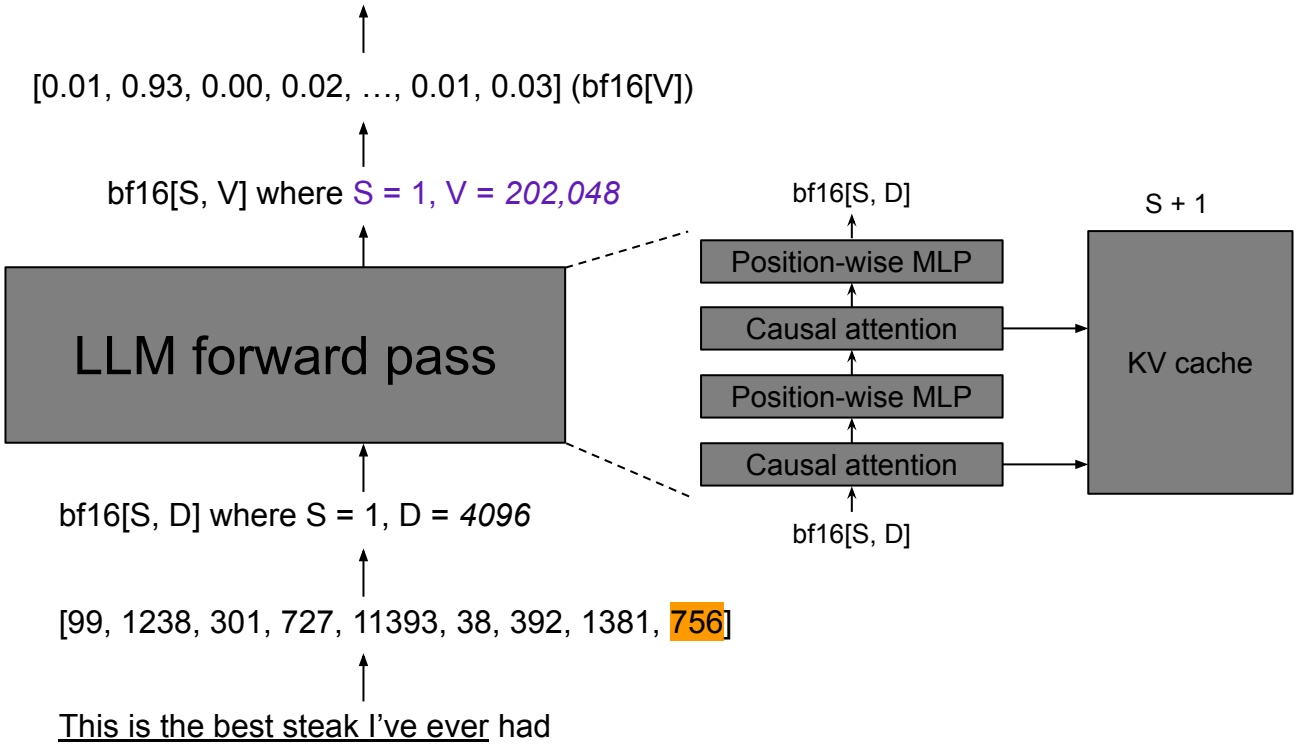
Prefill: The First Token



Decode: Second+ tokens



Decode: Second+ tokens



Decode: Second+ tokens

[0.01, 0.93, 0.00, 0.02, …, 0.01, 0.03] (bf16[V])

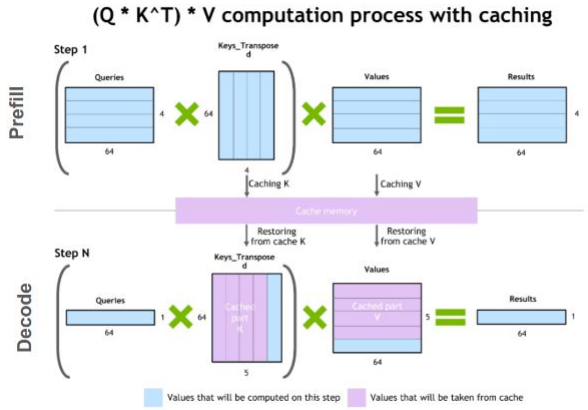
bf16[S, V] where S = 1, V = *202,048*

LLM forward pass

bf16[S, D] where S = 1, D = *4096*

[99, 1238, 301, 727, 11393, 38, 392, 1381, 756]

This is the best steak I've ever had



Agenda

- LLM Inference
  - KV Cache
  - Prefill & Decode
- Performance Metrics
  - TTFT, TTLT, TBT, TPOT
  - Arithmetic Intensity
- Optimizations
  - Orca (OSDI ’22)
  - vLLM (SOSP ’23)
- More System Challenges
  - Evaluating LLM Inference Systems
- Please ask questions if you don’t get something

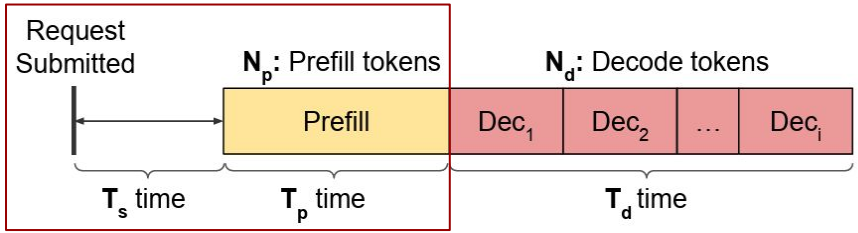
KV Cache

- Characteristics
  - Constant size for **each token in the context** (input or output)
  - So, for each output token generated, KV cache size grows **linearly**
- Size calculation for a single token
  - (# Transformer layers) x (# KV heads) x 2 (K and V) x (head output dimension) x 2 bytes

Model	1 token	4096 tokens
Llama 3.1 8B	32 x 8 x 2 x 128 x 2 bytes = 128 KiB	0.50 GiB
Llama 3.1 70B	80 x 8 x 2 x 128 x 2 bytes = 320 KiB	1.25 GiB
Llama 3.1 405B	126 x 8 x 2 x 128 x 2 bytes = 504 KiB	1.97 GiB

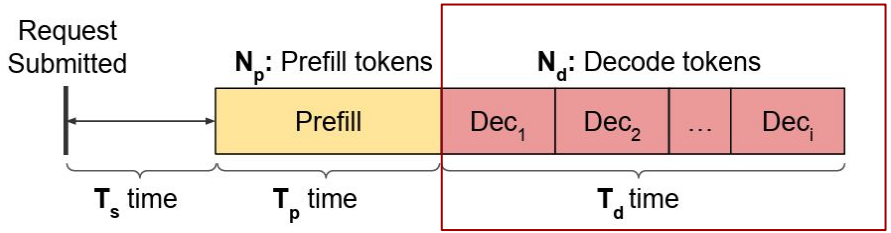
Optimization Metrics

- Prefill latency
  - **Time To First Token (TTFT)**: (T_s + T_p) time
  - TTFT + queuing delay is the user-experienced wait time



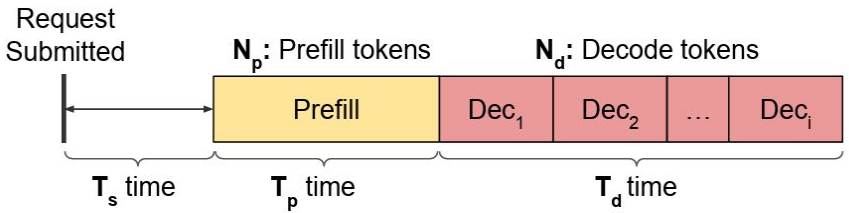
Optimization Metrics

- Prefill latency
  - **Time To First Token (TTFT)**: (T_s + T_p) time
  - TTFT + queuing delay is the user-experienced wait time
- Decode latency
  - **Time to Last Token (TTLT)**: (T_s + T_p + T_d) time
  - **Time Per Output Token (TPOT)**: Average decode latency for all output tokens - (T_d / N_d) time
  - **Time Between Tokens (TBT)**: Latencies between each output token - [T_d^1, T_d^2, …, T_d^i] time
  - Long decode latency leads to a laggy chat experience and will annoy users
  - It also doesn't have to be infinitely fast if the LLM is user-facing (6 tokens/second reading speed)



Optimization Metrics

- Prefill latency
  - **Time To First Token (TTFT)**: (T_s + T_p) time
  - TTFT + queuing delay is the user-experienced wait time
- Decode latency
  - **Time to Last Token (TTLT)**: (T_s + T_p + T_d) time
  - **Time Per Output Token (TPOT)**: Average decode latency for all output tokens - (T_d / N_d) time
  - **Time Between Tokens (TBT)**: Latencies between each output token - [T_d^1, T_d^2, …, T_d^i] time
  - Long decode latency leads to a laggy chat experience and will annoy users
  - It also doesn't have to be infinitely fast if the LLM is user-facing (6 tokens/second reading speed)
- Throughput
  - Request throughput (reqs/s)



A Potential Optimization: Batching

- Training had batch size; what about for inference?
  - Running more than one requests together
- LLMs are pretty weird
  - Very large in size
  - Prefill and decode
- Questions
  - Is batching a no-brainer optimization, or does it only sometimes make sense?
  - How should we *reason* about batching for LLMs on modern GPUs?

Matrix Multiplication

- It's everywhere in Transformers and takes up most of the compute
  - Some in attention, and the MLP is basically two *big* matmuls
- Focus: Position-wise MLP
  - Batch size (B) is the total number of tokens being forwarded through the MLP
  - Say we batched together three requests with input sequence length: 3, 4, and 5

Input	bf16[B, D]
Weight	bf16[D, F]
Output	bf16[B, F]

Matrix Multiplication

- It's everywhere in Transformers and takes up most of the compute
  - Some in attention, and the MLP is basically two *big* matmuls
- Focus: Position-wise MLP
  - Batch size (B) is the total number of tokens being forwarded through the MLP
  - Say we batched together three requests with input sequence length: 3, 4, and 5
  - Prefill: B = 12 – total number of input tokens
  - Decode: B = 3 – number of tokens generated (== number of requests)

Input	bf16[B, D]
Weight	bf16[D, F]
Output	bf16[B, F]

Analyzing Matrix Multiplication

- Computation and memory accesses
  - B = 128, D = 4096, F = 16384

Input	bf16[B, D]
Weight	bf16[D, F]
Output	bf16[B, F]

	Computation	Memory
Matmul	2BDF = 16 x 10^9 FLOPs	2BD + 2DF + 2BF = 0.13 x 10^9 Bytes

Analyzing Matrix Multiplication

- Computation and memory accesses
  - B = 128, D = 4096, F = 16384

Input	bf16[B, D]
Weight	bf16[D, F]
Output	bf16[B, F]

	Computation	Memory
Matmul	2BDF = 16 x 10^9 FLOPs	2BD + 2DF + 2BF = 0.13 x 10^9 Bytes
H100 spec	1979 TFLOPs/s / 2 = 989.5 x 10^{12} FLOPs/s	3.35 TB/s = 3.35 x 10^{12} Bytes/s

Which one is bottlenecking our matmul?
***Computation throughput**, or **memory throughput**?*

Arithmetic Intensity

- A single number that tells us which one we're bottlenecked by
- **Arithmetic Intensity**: Computation per byte of memory access
  - Amount of computation / amount of memory access (FLOPs/bytes)

	Computation	Memory
Matmul	2BDF = 16 x 10^9 FLOPs	2BD + 2DF + 2BF = 0.13 x 10^9 Bytes
H100 spec	1979 TFLOPs/s / 2 = 989.5 x 10^{12} FLOPs/s	3.35 TB/s = 3.35 x 10^{12} Bytes/s

Which one is bottlenecking our matmul?
***Computation throughput**, or **memory throughput**?*

Arithmetic Intensity

- A single number that tells us which one we're bottlenecked by
- **Arithmetic Intensity**: Computation per byte of memory access
  - Amount of computation / amount of memory access (FLOPs/bytes)

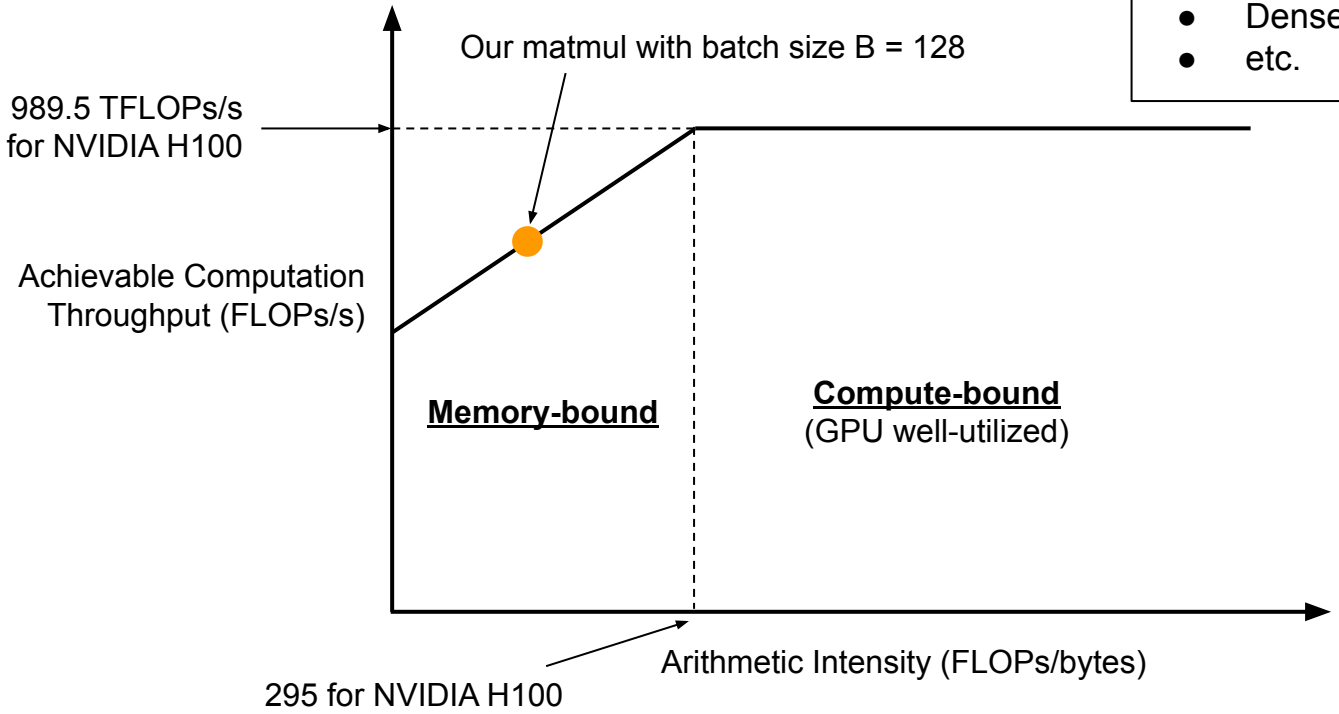
	Computation	Memory	Computation per byte
Matmul	2BDF = 16 x 10^9 FLOPs	2BD + 2DF + 2BF = 0.13 x 10^9 Bytes	Exactly: 123 **Approximate with B**: 128
H100 spec	1979 TFLOPs/s / 2 = 989.5 x 10^{12} FLOPs/s	3.35 TB/s = 3.35 x 10^{12} Bytes/s	Exactly: 295

B = 128, D = 4096, F = 16384

*It's **memory bound**.*

The Roofline Plot

You get a different roofline for each
- Hardware
- Floating point precision
- Dense vs. sparse operation
- etc.



What All This Means for Prefill and Decode

- Position-wise MLP
  - Batch size (B) is the total number of tokens being forwarded through the MLP
  - Say we batched together three requests with input sequence length: 3, 4, and 5
  - Prefill: B = 12 – total number of input tokens
  - Decode: B = 3 – number of tokens generated (== number of requests)
- Prefill and Decode
  - **Prefill**: 295 tokens across input sequences is easy, so you're **nearly always compute-bound**
    - You actually don't have to run multiple *requests* together for prefill to be compute-bound

What All This Means for Prefill and Decode

- Position-wise MLP
  - Batch size (B) is the total number of tokens being forwarded through the MLP
  - Say we batched together three requests with input sequence length: 3, 4, and 5
  - Prefill: B = 12 – total number of input tokens
  - Decode: B = 3 – number of tokens generated (== number of requests)
- Prefill and Decode
  - **Prefill**: 295 tokens across input sequences is easy, so you're **nearly always compute-bound**
    - You actually don't have to run multiple *requests* together for prefill to be compute-bound
  - **Decode**: Running 295 *requests* together is *not easy*, so you're **usually memory-bound**
    - We always want to **increase** the number of *requests* running together for decode

What All This Means for Prefill and Decode

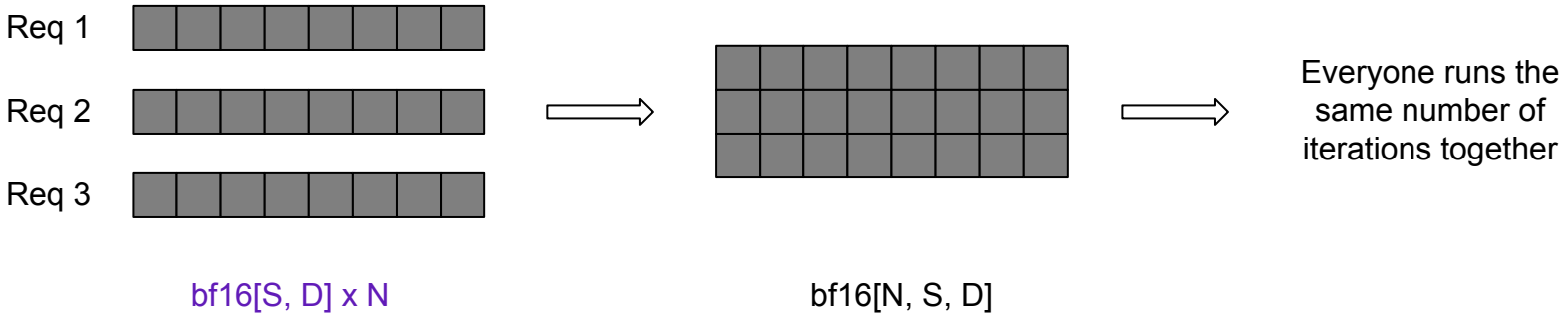
- Position-wise MLP
  - Batch size (B) is the total number of tokens being forwarded through the MLP
  - Say we batched together three requests with input sequence length: 3, 4, and 5
  - Prefill: B = 12 – total number of input tokens
  - Decode: B = 3 – number of tokens generated (== number of requests)
- Prefill and Decode
  - **Prefill**: 295 tokens across input sequences is easy, so you're **nearly always compute-bound**
    - You actually don't have to run multiple *requests* together for prefill to be compute-bound
  - **Decode**: Running 295 *requests* together is *not easy*, so you're **usually memory-bound**
    - We always want to **increase** the number of *requests* running together for decode
- What about parallelism?
  - DP: Serving with more replicas
  - PP: Pipeline schedule becomes more complex with token-level generation and unpredictable length
  - TP: Communication may become the bottleneck

Agenda

- LLM Inference
  - KV Cache
  - Prefill & Decode
- Performance Metrics
  - TTFT, TPOT, etc
  - Arithmetic Intensity
- Optimizations
  - Orca (OSDI ’22)
  - vLLM (SOSP ’23)
- More System Challenges
  - Evaluating LLM Inference Systems
- Please ask questions if you don’t get something

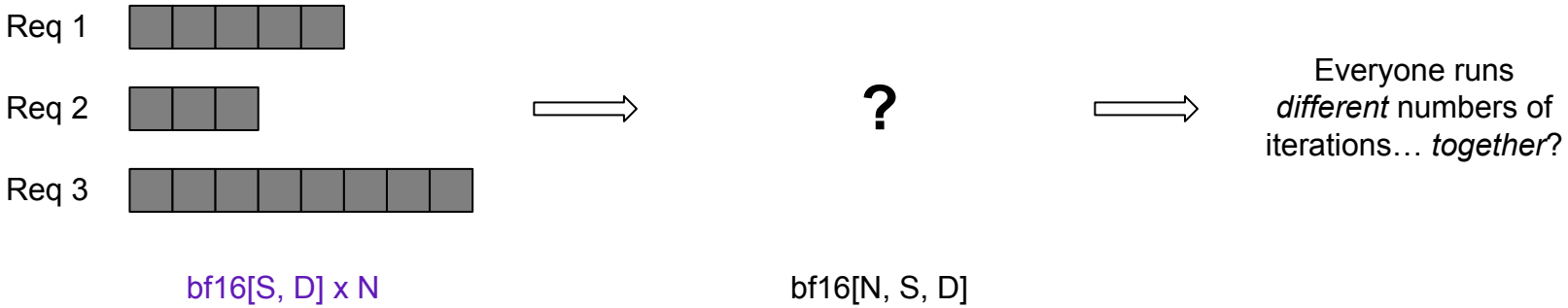
Orca (OSDI '22): Efficient Batching for LLMs

- If LLM inference requests
  - had the same number of input tokens, and
  - had the same number of output tokens (i.e., same number of decode iterations),
  - then batching is trivial



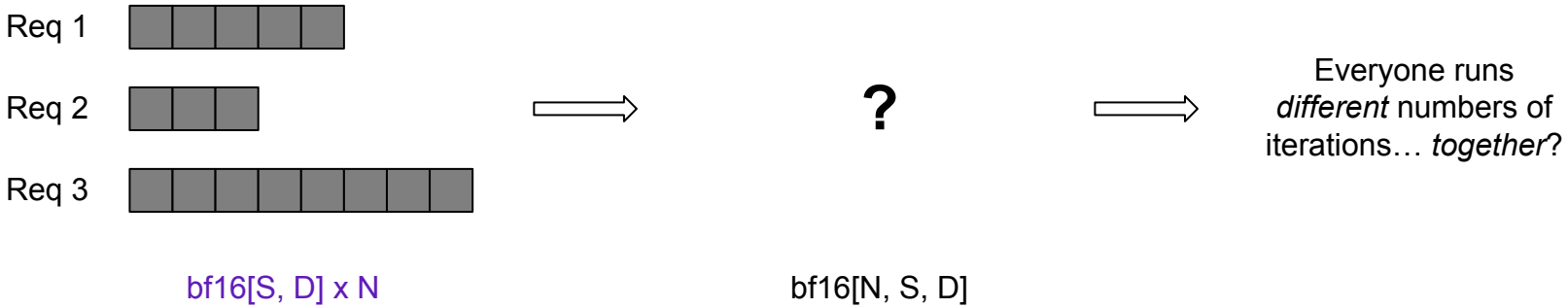
Orca (OSDI '22): Efficient Batching for LLMs

- If LLM inference requests
  - had the same number of input tokens, and
  - had the same number of output tokens (i.e., same number of decode iterations),
  - then batching is trivial



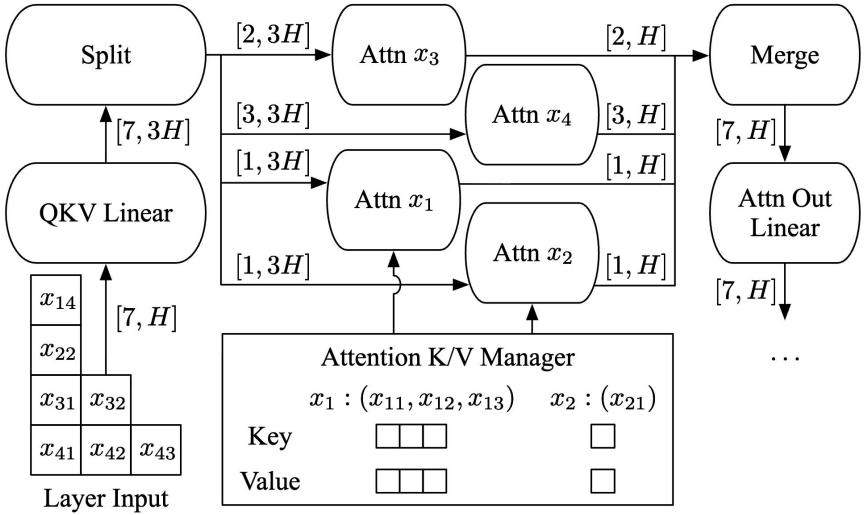
Orca (OSDI '22): Efficient Batching for LLMs

- Position-wise MLP
  - This is position-wise anyway
  - Just flatten all the tokens together into the batch dimension, and run forward



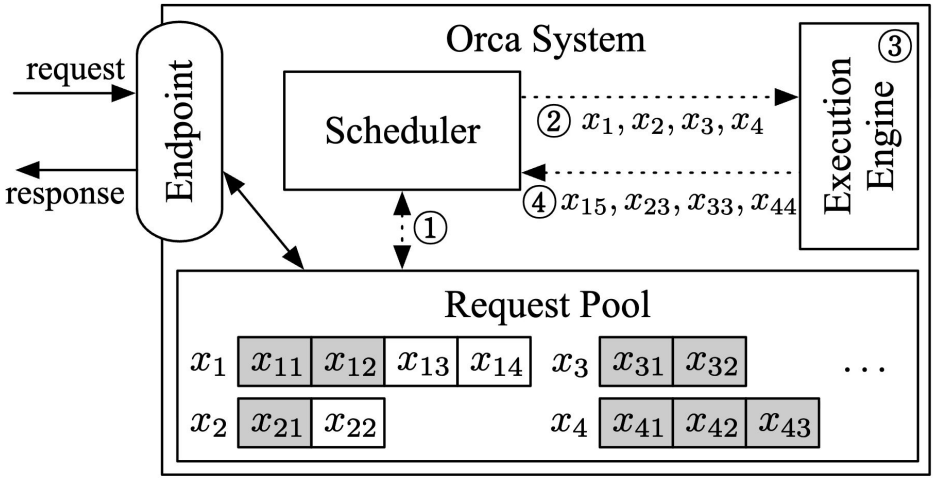
Orca (OSDI '22): Efficient Batching for LLMs

- Attention
  - GPUs are parallel enough
  - Just dispatch the attention operation of each sequence in parallel



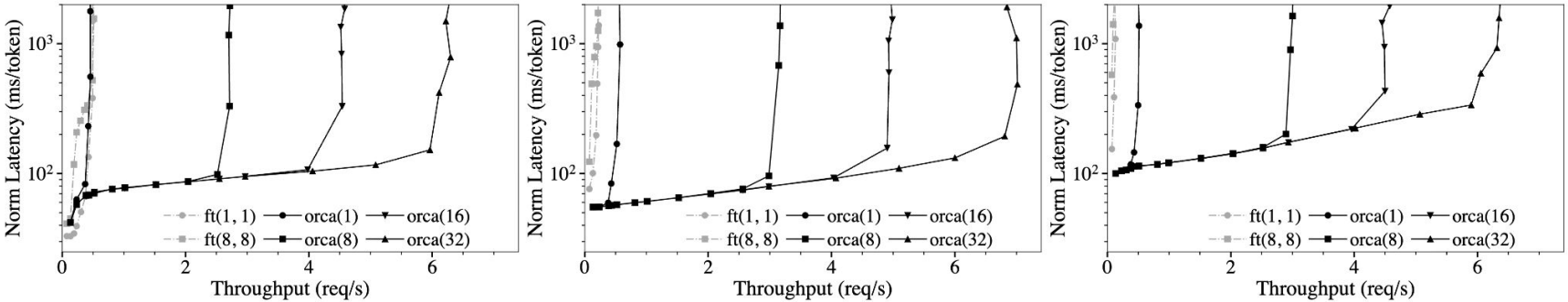
Orca (OSDI '22): Efficient Batching for LLMs

- Scheduling benefits
  - Requests may enter and leave the system at iteration boundaries
  - Run one iteration → Back to scheduler, finished ones leave the system
  - New request → Enqueued into scheduler, can start processing when there's an empty slot



Orca (OSDI '22): Efficient Batching for LLMs

- Crushes baselines
  - NVIDIA A100-40GB GPUs
  - Batch size is in the order of *tens*

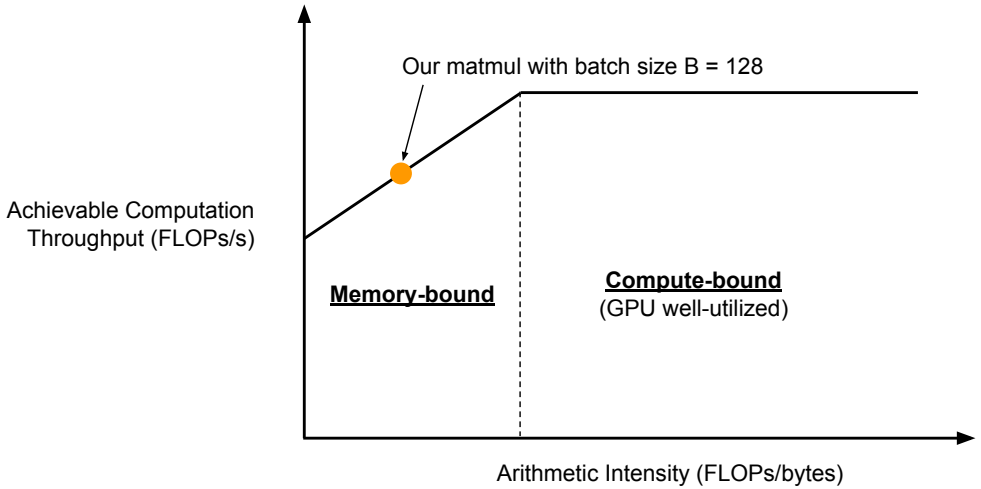


(a) 101B model, 8 GPU. (b) 175B model, 16 GPUs. (c) 341B model, 32 GPUs.

Figure 10: Median end-to-end latency normalized by the number of generated tokens and throughput. Label "orca(*max_bs*)" represents results from ORCA with a max batch size of *max_bs*. Label "ft(*max_bs*, *mbs*)" represents results from FasterTransformer with a max batch size of *max_bs* and a microbatch size of *mbs*.

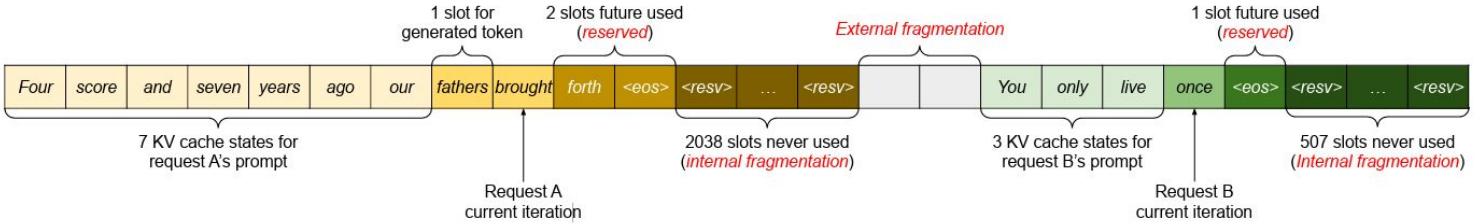
vLLM (SOSP '23): Memory is Key!

- Recall arithmetic intensity of matrix multiplication
  - We wanted a **larger batch size** for better hardware utilization & throughput
- Why can't we increase batch size very easily for Decode?
  - Each request needs to have its **KV cache** in GPU memory – hits **VRAM capacity**



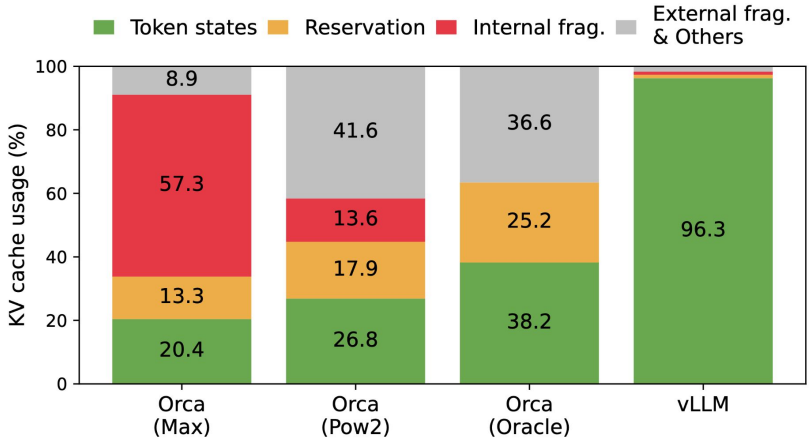
vLLM (SOSP '23): Memory is Key!

- You don't know how much KV cache memory to allocate for a request
  - Because you don't know **how many output tokens** it'll generate
- KV cache memory pre-allocation leads to memory wastage



vLLM (SOSP '23): Memory is Key!

- You don't know how much KV cache memory to allocate for a request
  - Because you don't know **how many output tokens** it'll generate
- KV cache memory pre-allocation leads to memory wastage
  - Max, Pow2 (Half-oracle), and Oracle: Policies for how much memory is pre-allocated
  - In all cases, memory is pre-allocated during the **entire lifetime** of the request

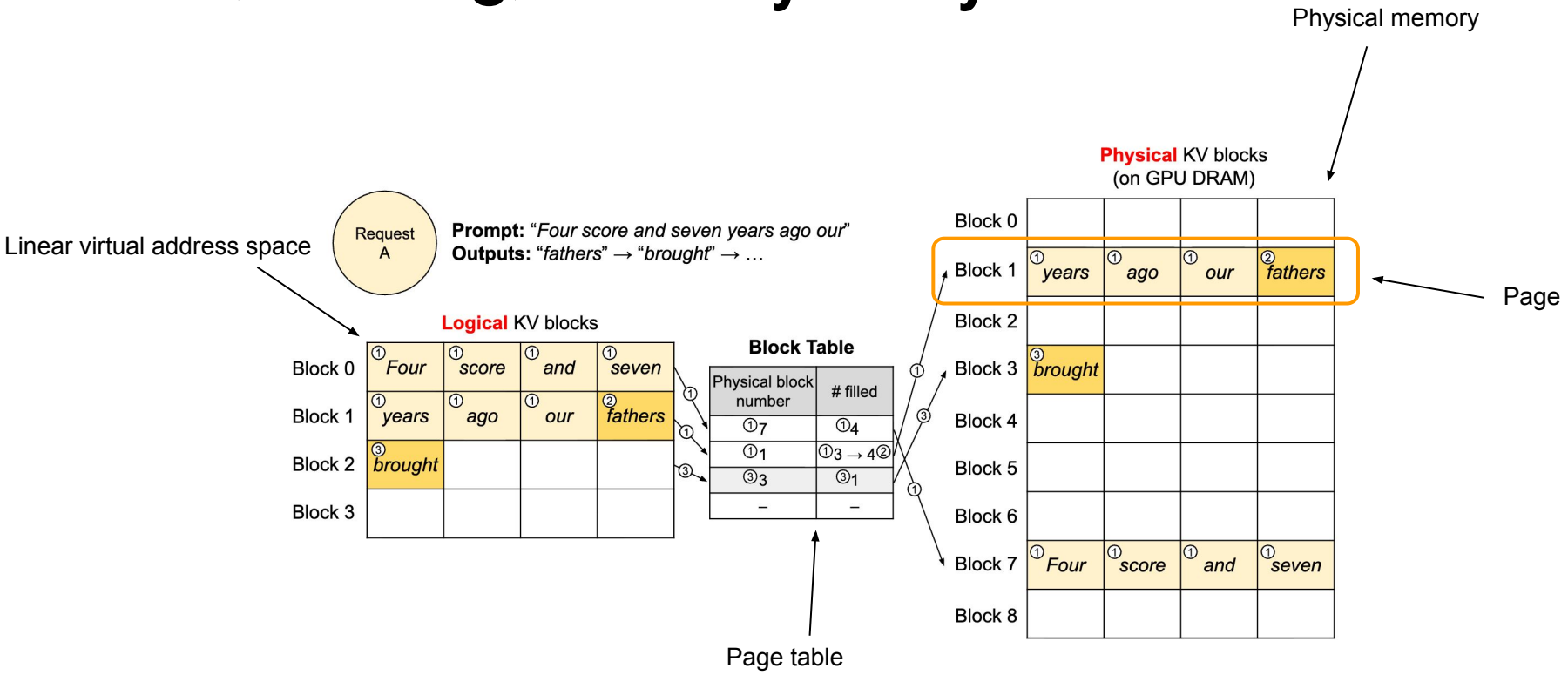


vLLM (SOSP '23): Memory is Key!

- This sounds familiar
  - There are units of work, and they come and go
  - You don't know how much memory these units will consume (they can vary quite a bit)
  - When the unit goes away, you deallocate all memory associated with it
  - Like… *processes*

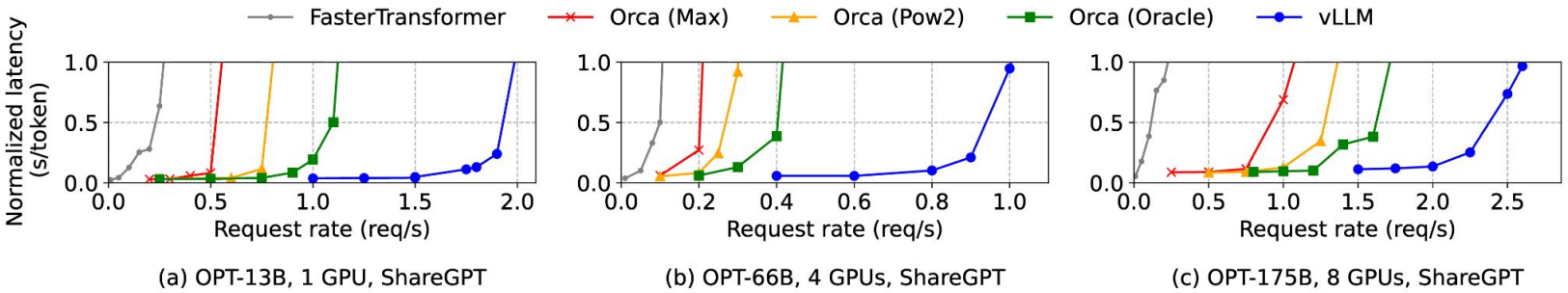
Virtual memory and pages

vLLM (SOSP '23): Memory is Key!



vLLM (SOSP '23): Memory is Key!

- Crushes baselines
  - NVIDIA A100-40GB and A100-80GB GPUs
  - For each system, uses the largest batch size that fits in memory



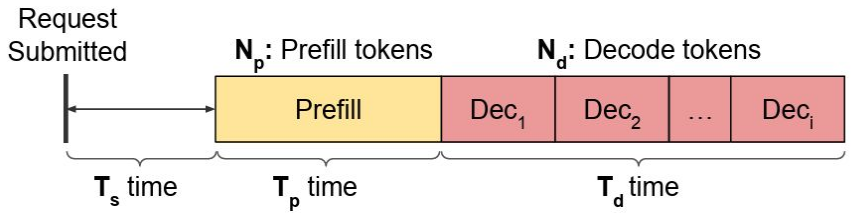
(a) OPT-13B, 1 GPU, ShareGPT (b) OPT-66B, 4 GPUs, ShareGPT (c) OPT-175B, 8 GPUs, ShareGPT

Agenda

- LLM Inference
  - KV Cache
  - Prefill & Decode
- Performance Metrics
  - TTFT, TPOT, etc
  - Arithmetic Intensity
- Optimizations
  - Orca (OSDI ’22)
  - vLLM (SOSP ’23)
- More System Challenges
  - Evaluating LLM Inference Systems
- Please ask questions if you don’t get something

Evaluating LLM Inference Systems

- Prefill latency
  - **Time To First Token (TTFT)**: (T_s + T_p) time
  - TTFT + queuing delay is the user-experienced wait time
- Decode latency
  - **Time to Last Token (TTLT)**: (T_s + T_p + T_d) time
  - **Time Per Output Token (TPOT)**: Average decode latency for all output tokens - (T_d / N_d) time
  - **Time Between Tokens (TBT)**: Latencies between each output token - [T_d^1, T_d^2, …, T_d^i] time
  - Long decode latency leads to a laggy chat experience and will annoy users
  - It also doesn't have to be infinitely fast if the LLM is user-facing
- Throughput
  - Request throughput (reqs/s)



Evaluating LLM Inference Systems

- Implementation Fairness
  - DistServe vs NanoFlow
  - Are baselines implemented comparably?
  - If not, are microbenchmarks or ablations used to isolate algorithmic gains from system implementation differences?

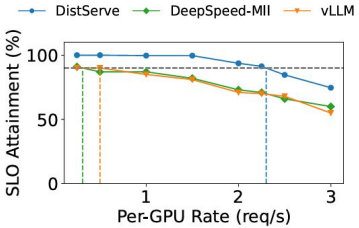


Figure 8: Chatbot application with OPT models on the ShareGPT dataset.

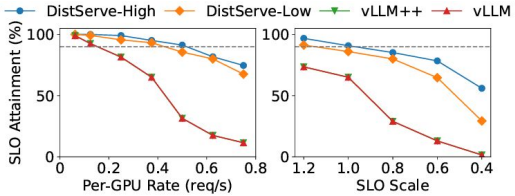
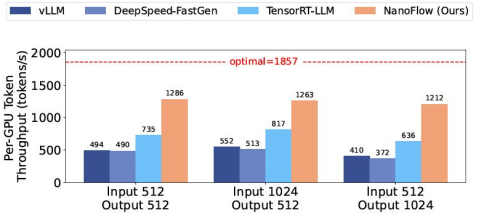


Figure 11: Ablation experiments.



(a) LLaMA-2-70B, 8 GPU, TP=8, Constant Input & Output Length

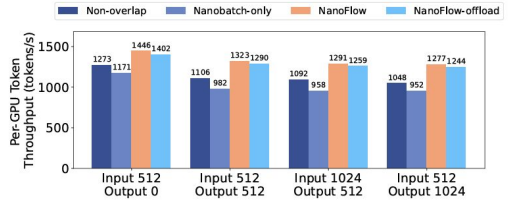


Figure 9: Ablation study results for NanoFlow. Nano-batching and overlapping improves NanoFlow's performance.

Evaluating LLM Inference Systems

- Parameter Tuning
  - DistServe vs Sarathi-Serve
  - Are all performance-critical configuration parameters documented for both the proposed system and baselines?
  - Were baseline parameters tuned appropriately for the evaluation setup?

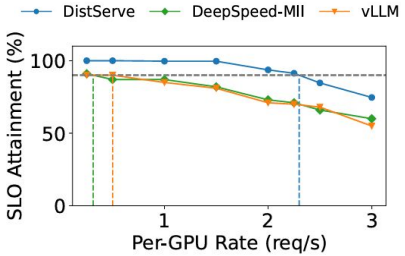


Figure 8: Chatbot application with OPT models on the ShareGPT dataset.

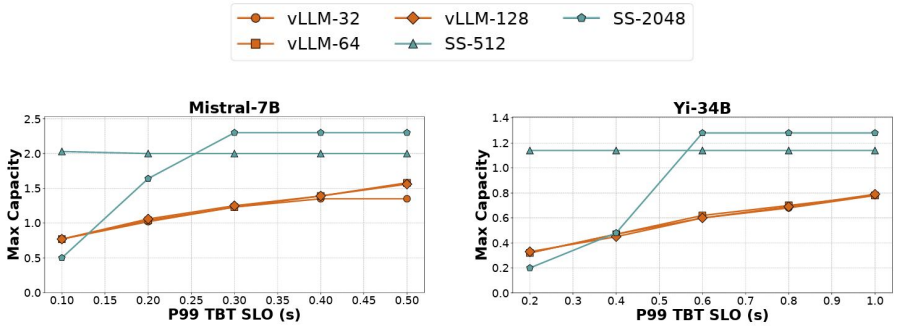


Figure 12: Latency – Throughput tradeoff in vLLM and Sarathi-Serve for Mistral-7B and Yi-34B models on *openchat_sharegpt4* dataset. We evaluate vLLM with three different max batch sizes of 32, 64 and 128. For Sarathi-Serve, we consider token budget of 512 and 2048 with max batch size of 128. Sarathi-Serve delivers 3.5× higher capacity under stringent SLOs for Yi-34B using *Stall-free batching*.

Evaluating LLM Inference Systems

- Model Selection
  - vLLM vs Sarathi-Serve
  - Do the evaluated models reflect current state-of-the-art?
  - Are key techniques evaluated across different model architectures where their impact might significantly vary?

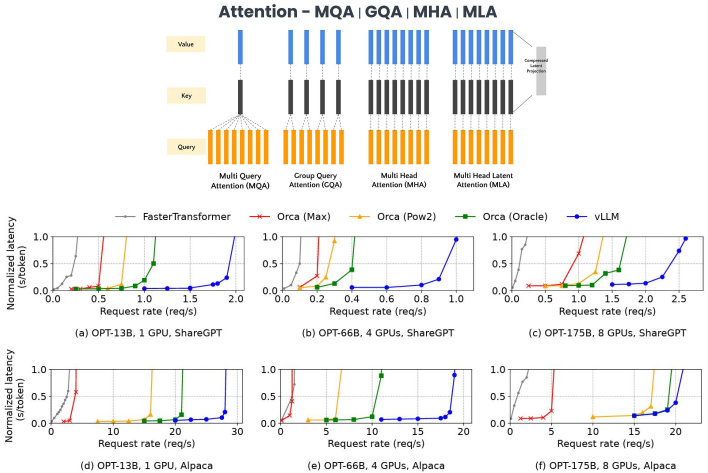


Figure 12. Single sequence generation with OPT models on the ShareGPT and Alpaca dataset

Model	Attention Mechanism	GPU Configuration	Memory Total (per-GPU)
Mistral-7B	GQA-SW	1 A100	80GB (80GB)
Yi-34B	GQA	2 A100s (TP2)	160GB (80GB)
LLaMA2-70B	GQA	8 A40s (TP4-PP2)	384GB (48GB)
Falcon-180B	GQA	4 A100s×2 nodes (TP4-PP2)	640GB (80GB)

Table 1: Models and GPU configurations (GQA: grouped-query attention, SW: sliding window).

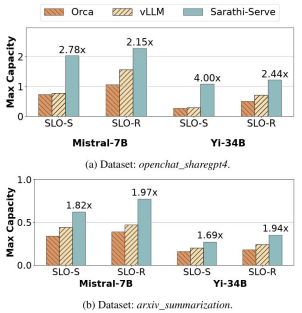


Figure 10: Capacity (in queries per second) of Mistral-7B and Yi-34B with different schedulers under strict (SLO-S) and relaxed (SLO-R) latency SLOs.

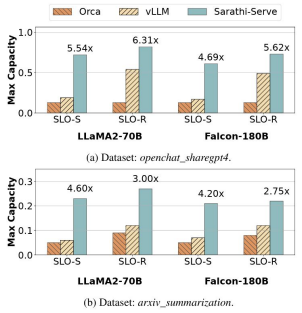
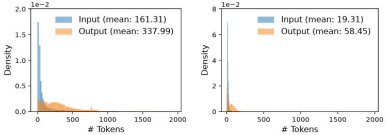


Figure 11: Capacity of LLaMA2-70B and Falcon-180B (models with pipeline parallelism) with different schedulers under strict (SLO-S) and relaxed (SLO-R) latency SLOs.

Evaluating LLM Inference Systems

- Workload Diversity
  - vLLM vs Sarathi-Serve
  - Is the system evaluated beyond a single dataset or synthetic traces?
  - Are diverse applications (chat, code, summarization) with their characteristic distributions of prompt/output lengths and interaction models evaluated?



(a) ShareGPT (b) Alpaca

Figure 11. Input and output length distributions of the (a) ShareGPT and (b) Alpaca datasets.

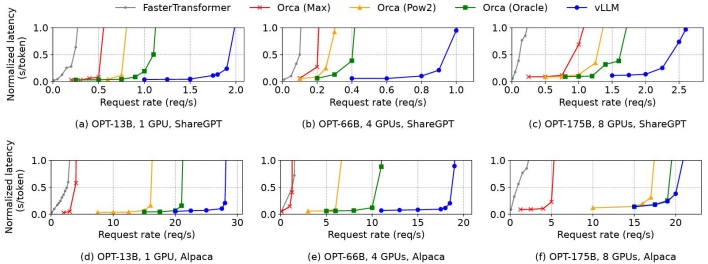


Figure 12. Single sequence generation with OPT models on the ShareGPT and Alpaca dataset

Dataset	Prompt Tokens			Output Tokens		
	Median	P90	Std.	Median	P90	Std.
openchat_sharegpt4	1730	5696	2088	415	834	101
arxiv_summarization	7059	12985	3638	208	371	265

Table 2: Datasets used for evaluation.

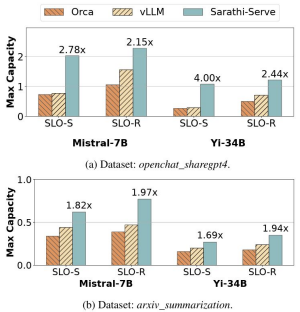


Figure 10: Capacity (in queries per second) of Mistral-7B and Yi-34B with different schedulers under strict (SLO-S) and relaxed (SLO-R) latency SLOs.

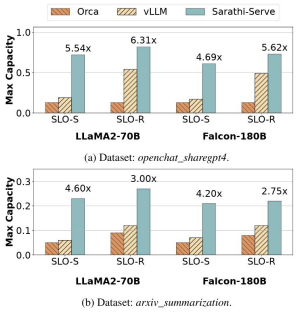


Figure 11: Capacity of LLaMA2-70B and Falcon-180B (models with pipeline parallelism) with different schedulers under strict (SLO-S) and relaxed (SLO-R) latency SLOs.

Evaluating LLM Inference Systems

- Practical Latency
  - DistServe vs LoongServe
  - Are reported improvements achieved within latency bounds (e.g., TTFT, TBT SLOs) suitable for the target application(s)?
  - Is the connection between reported metrics and absolute user-experienced latency clear?

Application	Model Size	TTFT	TPOT	Dataset
Chatbot OPT-13B	26GB	0.25s	0.1s	ShareGPT [8]
Chatbot OPT-66B	132GB	2.5s	0.15s	ShareGPT [8]
Chatbot OPT-175B	350GB	4.0s	0.2s	ShareGPT [8]
Code Completion OPT-66B	132GB	0.125s	0.2s	HumanEval [14]
Summarization OPT-66B	132GB	15s	0.15s	LongBench [13]

Table 1: Workloads in evaluation and latency requirements.

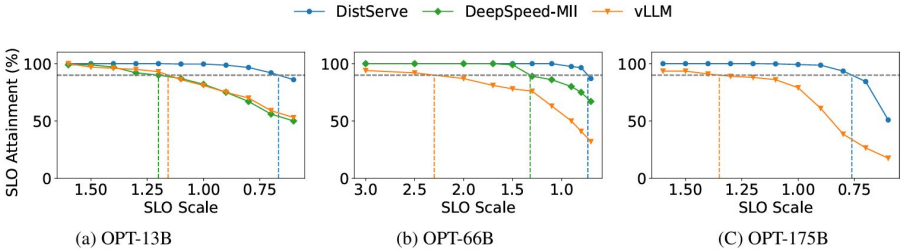


Figure 8: Chatbot application with OPT models on the ShareGPT dataset.

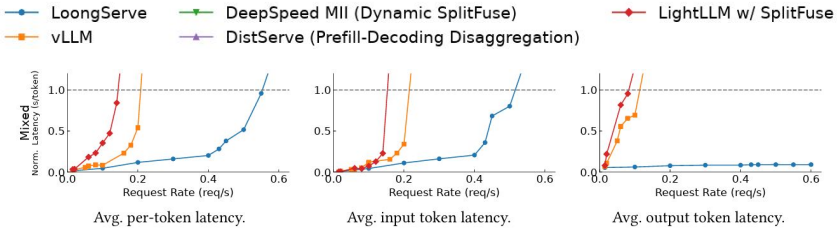


Figure 10. Average latency of different LLM serving systems with the LWM-1M-Text (Llama-2-7B) on real-world workloads.

Evaluating LLM Inference Systems

- Metric Selection
  - dLoRA vs DistServe
  - Are the chosen metrics broadly used and understood?
  - For a novel metric, is its relationship to user-perceived performance explained and justified?

Metrics. We use the average latency as the primary metric to evaluate the performance of dLoRA. Following prior work on LLM serving [11, 12], the average latency is calculated by dividing the sum of each request's end-to-end latency by the total number of output tokens.

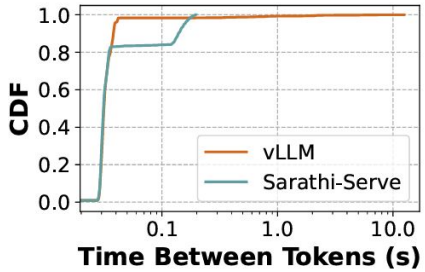
$$\text{NL}_{\text{global}} = \frac{\sum_{i=1}^{N} L_i}{\sum_{i=1}^{N} T_i}$$

Key metrics. We focus on serving throughput. Specifically, using the workloads with different request rates, we measure *normalized latency* of the systems, the mean of every request's end-to-end latency divided by its output length, as in Orca [60]. A high-throughput serving system should retain low normalized latency against high request rates.

$$\text{NL}_{\text{per-req}} = \frac{1}{N} \sum_{i=1}^{N} \frac{L_i}{T_i}$$

Evaluating LLM Inference Systems

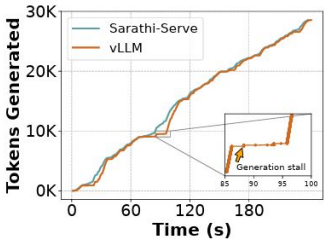
- Performance Distribution
  - vLLM vs Sarathi-Serve
  - Does the evaluation present the full latency distribution (e.g., CDFs) or at least key percentiles (P50, P90, P99), rather than just a single summary statistic like average or median?
  - Does the analysis illuminate performance trade-offs across different points in the distribution?



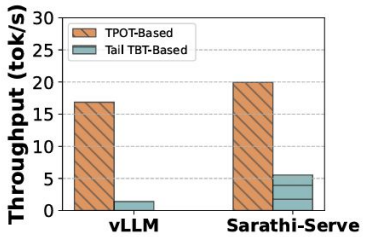
(c) Decode token latency distribution.

Evaluating LLM Inference Systems

- No Obscuring Normalization
  - dLoRA vs DistServe
  - When using normalized metrics (Normalized Latency, TPOT), are raw performance characteristics also examined?
  - Are critical temporal aspects like scheduling delays (Ts) and token generation stalls (TBT variance) analyzed independently?
  - Does the evaluation consider how normalization might obscure user-facing performance issues evident in raw measurements?



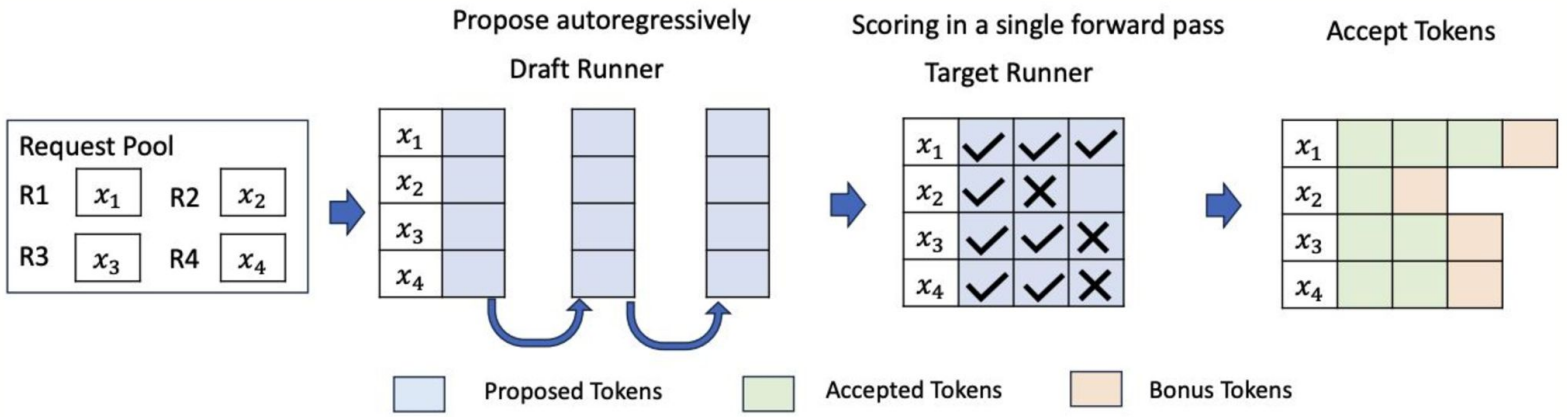
(a) Generation stalls in output tokens generation.



(b) Token throughput derived using various latency metrics.

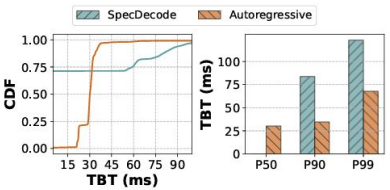
Evaluating LLM Inference Systems

- Case Study - Speculative Decoding Systems
  - A smaller ”draft” model predicts multiple future tokens
  - Larger primary model verifies the predicted tokens in a single forward pass



Evaluating LLM Inference Systems

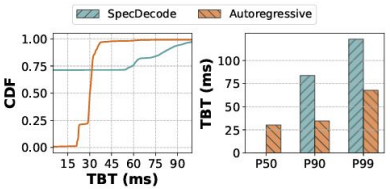
- Case Study - Speculative Decoding Systems
  - Successful verifications generate multiple tokens almost instantly
  - Failed verification failures lead to delays as only a single token is produced
    - Incurring costs for both draft generation and verification
  - TBT becomes misleading



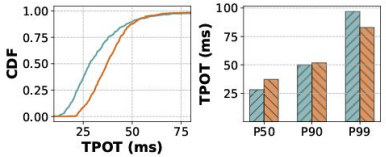
(a) TBT latency characteristics showing bursty token generation in speculative decoding. CDF (left) reveals 75% tokens with near-zero TBT due to batch acceptance.

Evaluating LLM Inference Systems

- Case Study - Speculative Decoding Systems
  - Successful verifications generate multiple tokens almost instantly
  - Failed verification failures lead to delays as only a single token is produced
    - Incurring costs for both draft generation and verification
  - TBT becomes misleading
  - TPOT shows conflicting trends



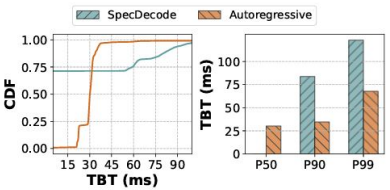
(a) TBT latency characteristics showing bursty token generation in speculative decoding. CDF (left) reveals 75% tokens with near-zero TBT due to batch acceptance.



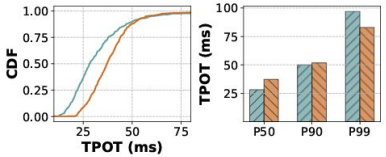
(b) TPOT analysis showing the aggregate latency trends: lower median costs but higher tail latency.

Evaluating LLM Inference Systems

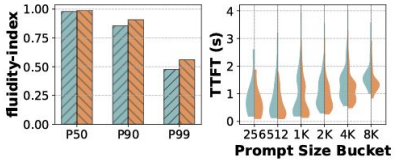
- Case Study - Speculative Decoding Systems
  - Successful verifications generate multiple tokens almost instantly
  - Failed verification failures lead to delays as only a single token is produced
    - Incurring costs for both draft generation and verification
  - TBT becomes misleading
  - TPOT shows conflicting trends
  - TTFT is negatively impacted



(a) TBT latency characteristics showing bursty token generation in speculative decoding. CDF (left) reveals 75% tokens with near-zero TBT due to batch acceptance.



(b) TPOT analysis showing the aggregate latency trends: lower median costs but higher tail latency.



(c) Fluidity-index values at different percentiles (left) and TTFT distributions across prompt sizes (right), showing higher TTFT for speculative decoding due to additional prefill overhead for drafter.

In the Coming Weeks

- You will learn about
  - Different workloads
  - Parallelism
  - Disaggregation
  - Resource management
  - Optimization metric design
  - Various sources of heterogeneity
  - … and more!
- Thank you!

(Unofficial) Textbooks

- How to Scale Your Model (Google DeepMind)
  - Theoretical analysis of computations that are important to LLMs
  - Arithmetic intensity, compute- and memory-bound, roofline, back-of-the-envelope estimations
- The Ultra-Scale Playbook (Hugging Face)
  - Training LLMs on GPU Clusters
  - Various types of training parallelisms, implications on compute, memory, and communication
- Use as references
  - Each thing will take at least a week of full-time reading (it's worth it, though)
  - It's a fast-evolving field; the only way to be relevant is to continuously read